

The goal was to redesign the system so that the app could retrieve all required data in under 2.5 seconds while keeping the data fresh and correctly handling inter-service dependencies.

A Backend for Frontend (BFF) was introduced to handle data aggregation on behalf of the mobile app. Instead of assembling data at request time, the BFF maintained a pre-computed, denormalized read model that was continuously updated using an event-driven approach. Each microservice published events when its data changed, and the BFF consumed these events to update the aggregated view. This shifted dependency resolution and data preparation from runtime to background processing.

- Add a Backend for Frontend (BFF) so the mobile app calls only one service
- Move data aggregation logic from the mobile app to the backend
- Pre-compute the first-page data instead of building it on every request
- Use event-driven architecture so services publish data changes
- Maintain a denormalized read model / materialized view for the first screen
- Update the read model whenever any of the 15 services changes data
- Resolve service dependencies ahead of time, not during request processing
- Let the mobile app fetch all required data with a single fast request
- Ensure near real-time freshness without relying on long-lived caches
- Optionally split first-screen data into critical and non-critical data to improve perceived speed

The mobile app was reduced to making a single backend request, significantly lowering network overhead and response time. All required first-page data could be returned immediately from the pre-computed view, consistently meeting the 2.5-second performance target while maintaining data freshness and system scalability.